

1.2. NumPy-da massivlarni yaratish (Creating Arrays in NumPy)

1.2.1. NumPy massivlarni yaratish (Array Creation)

NumPy massivlari — Python-dagi ro'yxatlarga (list) o'xshash, ammo matematik va sonli amallar uchun maxsus optimallashtirilgan to'rsimon (grid-like) tuzilmalardir. NumPy massivini yaratishning eng oddiy yo'li — `np.array()` funksiyasi yordamida oddiy Python ro'yxatini massivga o'tkazishdir.

Buni quyidagi misol yordamida ko'rib chiqamiz:

```
import numpy as np

# Bir o'lchamli massiv (1D array)
# Oddiy sonlar ketma-ketligidan hosil qilinadi
massiv1 = np.array([1, 2, 3, 4, 5])
print("1-o'lchamli massiv:\n", massiv1)

# Ikki o'lchamli massiv (2D array)
# Ichma-ich joylashgan ro'yxatlar (qatorlar va ustunlar) yordamida hosil qil
massiv2 = np.array([[1, 2], [3, 4]])
print("\n2-o'lchamli massiv:\n", massiv2)
```

```
1-o'lchamli massiv:
[1 2 3 4 5]
```

```
2-o'lchamli massiv:
[[1 2]
 [3 4]]
```

Izoh:

- *Bir o'lchamli massiv:* Faqat bitta o'qdan (axis) iborat bo'lib, vektor shaklida bo'ladi.
- *Ikki o'lchamli massiv:* Qator va ustunlardan tashkil topgan matritsa ko'rinishida bo'lib, matematik amallarni bajarishda juda samarali hisoblanadi.

1.2.2. Maxsus qiymatlar bilan massivlar yaratish (Creating Arrays with Specific Values)

Maxsus qiymatlar bilan massivlar yaratish

Muayyan qiymatlarni massivga biriktirish uchun NumPy nollar, birlar yoki ixtiyoriy doimiy (konstanta) sonlar bilan to'ldirilgan massivlarni yaratish uchun bir nechta funksiyalarni taqdim etadi.

1. Nollar massivi (Zeros Array)

`np.zeros()` funksiyasi barcha elementlari 0 ga teng bo'lgan massiv yaratadi. Parametr sifatida massivning shaklini (o'lchamini) ko'rsatish talab etiladi.

```
# 3 qator va 4 ustundan iborat, nollar bilan to'ldirilgan massiv
nollar_massivi = np.zeros((3, 4))
print(nollar_massivi)
```

```
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
```

2. Birlar massivi (Ones Array)

`np.ones()` funksiyasi barcha elementlari 1 ga teng bo'lgan massiv hosil qiladi.

```
# 2 qator va 3 ustundan iborat, birlar bilan to'ldirilgan massiv
birlar_massivi = np.ones((2, 3))
print(birlar_massivi)
```

```
[[1. 1. 1.]
 [1. 1. 1.]]
```

3. To'ldirilgan massiv (Full Array)

`np.full()` funksiyasi massivni siz belgilagan maxsus bir xil qiymat bilan to'ldirish imkonini beradi.

```
# 2x2 o'lchamli, barcha elementlari 7 ga teng bo'lgan massiv
toldirilgan_massiv = np.full((2, 2), 7)
print(toldirilgan_massiv)
```

```
[[7 7]
 [7 7]]
```

Izoh:

- `np.zeros` va `np.ones` funksiyalari o'nli kasr (`float64`) turidagi sonlarni yaratadi (natijadagi nuqtaga e'tibor bering: `0.`).

- `np.full` funksiyasida esa ma'lumot turi siz kiritgan konstantaning turiga qarab belgilanadi.

1.2.3. Tasodifiy qiymatlar bilan massivlar yaratish (Creating Arrays with Random Values)

NumPy kutubxonasi tasodifiy sonlardan iborat massivlar yaratish uchun keng imkoniyatlarni taqdim etadi. Bu funksiyalar, ayniqsa, turli simulyatsiyalar, ma'lumotlarni testdan o'tkazish va algoritmlarni modellashtirishda juda foydali hisoblanadi.

1. *Tasodifiy o'nli kasrli massiv (Random Float Array)*

`np.random.rand()` funksiyasi 0 va 1 oralig'idagi tasodifiy o'nli kasr sonlardan iborat massiv hosil qiladi.

```
# 2 qator va 3 ustunli tasodifiy o'nli kasrlar massivi
tasodifiy_float = np.random.rand(2, 3)
print(tasodifiy_float)
```

```
[[0.56184844 0.43913504 0.90624475]
 [0.05254759 0.26191987 0.15927717]]
```

2. *Tasodifiy butun sonli massiv (Random Integers)*

Agar bizga ma'lum bir oraliqdagi butun sonlar kerak bo'lsa, `np.random.randint()` funksiyasidan foydalanamiz. Bunda biz sonlar oralig'ini va massiv o'lchamini (`size`) ko'rsatishimiz kerak.

```
# 1 dan 10 gacha bo'lgan (10 kirmaydi) oraliqdagi
# 3x3 o'lchamli tasodifiy butun sonlar massivi
tasodifiy_int = np.random.randint(1, 10, size=(3, 3))
print(tasodifiy_int)
```

```
[[4 3 7]
 [7 6 9]
 [6 6 6]]
```

Izoh:

- `np.random.rand()` funksiyasida o'lchamlar to'g'ridan-to'g'ri argument sifatida beriladi (`2, 3`).
- `np.random.randint(low, high, size)` funksiyasida `low` (quyi chegara) kiradi, lekin `high` (yuqori chegara) natijaga kirmaydi.

- Har safar kodni ishlatganingizda natijalar turlicha chiqadi, chunki sonlar tasodifiy generatsiya qilinadi.

1.2.4. Qiymatlar diapazoni bilan massivlar yaratish (Creating Arrays with a Range of Values)

Massivlarni yaratishning yana bir keng tarqalgan usuli — ma'lum bir sonlar oralig'idan foydalanishdir. NumPy bu maqsad uchun `np.arange()` va `np.linspace()` funksiyalarini taqdim etadi.

1. `np.arange()` funksiyasidan foydalanish

`np.arange()` funksiyasi berilgan oraliqda ma'lum bir qadam (interval) bilan joylashgan qiymatlardan massiv hosil qiladi. U Python-ning standart `range()` funksiyasiga juda o'xshaydi, lekin natija sifatida NumPy massivini (`ndarray`) qaytaradi.

```
# 0 dan 10 gacha bo'lgan, qadami 2 ga teng massiv  
# Eslatma: oxirgi chegara (10) massivga kirmaydi  
oralik_massiv = np.arange(0, 10, 2)  
print(oralik_massiv)
```

```
[0 2 4 6 8]
```

2. `np.linspace()` funksiyasidan foydalanish

`np.linspace()` funksiyasi belgilangan oraliqni siz ko'rsatgan miqdordagi teng bo'laklarga bo'lish orqali massiv yaratadi. `arange` dan farqi shundaki, bu yerda qadam hajmi emas, balki elementlar soni muhim hisoblanadi.

```
# 0 dan 1 gacha bo'lgan oraliqda aniq 5 ta teng masofadagi qiymat  
arr_linspace = np.linspace(0, 1, 5)  
print(arr_linspace)
```

```
[0.    0.25 0.5   0.75 1.   ]
```

Izoh:

- `np.arange`: Agar sizga har bir element orasidagi **masofa** (qadam) aniq bo'lishi kerak bo'lsa foydalaning.
- `np.linspace`: Agar sizga boshlang'ich va yakuniy nuqtalar orasida **necha element** bo'lishi kerakligi muhim bo'lsa foydalaning. Masalan,

grafiklar chizishda koordinata o'qlarini belgilash uchun `linspace` juda qulay.

1.2.5. Birlik va diagonal matritsalar (Identity and Diagonal Matrices)

NumPy chiziqli algebra masalalarida ko'p qo'llaniladigan **birlik matritsalar** va **diagonal matritsalar** yaratish uchun ham maxsus funksiyalarni taqdim etadi.

1. *Birlik matritsa (Identity Matrix)*

`np.eye()` funksiyasi birlik matritsani hosil qiladi. Bu kvadrat matritsa bo'lib, uning asosiy diagonal birlardan (1), qolgan barcha elementlari esa nollardan (0) iborat bo'ladi.

```
# 3x3 o'lchamli birlik matritsa yaratish
birlik_matritsa = np.eye(3)
print(birlik_matritsa)
```

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

2. *Diagonal matritsa (Diagonal Matrix)*

Berilgan elementlar ketma-ketligini matritsaning asosiy diagonaliga joylashtirish uchun `np.diag()` funksiyasidan foydalaniladi. Bunda diagonalga kirmaydigan barcha elementlar avtomatik ravishda 0 ga teng bo'ladi.

```
# [1, 5, 9] sonlarini asosiy diagonalga joylashtirgan holda matritsa yaratish
diagonal_matritsa = np.diag([1, 5, 9])
print(diagonal_matritsa)
```

```
[[1 0 0]
 [0 5 0]
 [0 0 9]]
```

Izoh:

- *Birlik matritsa* (`eye`) — asosan matritsalarini ko'paytirishda neytral element sifatida ishlatiladi (sonlardagi 1 kabi).
- *Diagonal matritsa* (`diag`) — ma'lum bir koeffitsiyentlarni o'zgaruvchilarga qo'llash yoki chiziqli tenglamalar tizimini soddalashtirishda juda qo'l keladi.

- E'tibor bering, `np.eye(3)` faqat bitta son (o'lcham) qabul qiladi, `np.diag([1, 2, 3])` esa diagonalda bo'lishi kerak bo'lgan elementlar ro'yxatini qabul qiladi.

1.2.6. NumPy-da massiv yaratish metodlari (Methods for Array Creation in NumPy)

NumPy-da massivlarni yaratish uchun ishlatiladigan asosiy funksiyalar va ularning tavsiflarini quyidagi jadval orqali ko'rib chiqamiz. Bu funksiyalar ma'lumotlar bilan ishlashda va hisoblash jarayonlarini optimallashtirishda poydevor vazifasini o'taydi.

NumPy massivlarini yaratish funksiyalari jadvali

Funksiya	Tavsifi (Izoh)
<code>empty()</code>	Berilgan shakl va turda, qiymatlari boshlang'ich holatga keltirilmagan yangi massiv qaytaradi.
<code>empty_like()</code>	Berilgan massiv bilan bir xil shakl va turga ega yangi "bo'sh" massiv qaytaradi.
<code>eye()</code>	Diagonali birlardan, qolgan qismlari nollardan iborat 2 o'lchamli massiv qaytaradi.
<code>identity()</code>	Kvadrat shaklidagi birlik matritsani qaytaradi.
<code>ones()</code>	Berilgan shakl va turda, barcha elementlari 1 bilan to'ldirilgan massiv qaytaradi.
<code>ones_like()</code>	Berilgan massiv shakli va turida, 1 lardan iborat massiv yaratadi.
<code>zeros()</code>	Berilgan shakl va turda, barcha elementlari 0 bilan to'ldirilgan massiv qaytaradi.
<code>zeros_like()</code>	Berilgan massiv shakli va turida, 0 lardan iborat massiv yaratadi.
<code>full_like()</code>	Berilgan massiv shakli va turida, ko'rsatilgan qiymat bilan to'ldirilgan massiv qaytaradi.

Funksiya	Tavsifi (Izoh)
<code>array()</code>	Ro'yxat yoki kortej kabi obyektlardan massiv yaratadi.
<code>asarray()</code>	Kiruvchi ma'lumotni massiv (<code>ndarray</code>) ko'rinishiga o'tkazadi.
<code>copy()</code>	Berilgan obyektning nusxasini (<code>copy</code>) massiv sifatida qaytaradi.
<code>fromiter()</code>	Takrorlanuvchi (<code>iterable</code>) obyektidan 1 o'lchamli yangi massiv hosil qiladi.
<code>arange()</code>	Berilgan oraliqda ma'lum bir qadam bilan joylashgan qiymatlarni qaytaradi.
<code>linspace()</code>	Berilgan oraliqda belgilangan miqdordagi teng masofali sonlarni qaytaradi.
<code>logspace()</code>	Logarifmik shkalada teng masofada joylashgan sonlarni qaytaradi.
<code>meshgrid()</code>	Koordinata vektorlaridan koordinata matritsalarini hosil qiladi.
<code>diag()</code>	Diagonal elementlarni ajratib oladi yoki diagonal massiv yaratadi.
<code>tril()</code>	Massivning pastki uchburchak qismini (<code>lower triangle</code>) qaytaradi.
<code>triu()</code>	Massivning yuqori uchburchak qismini (<code>upper triangle</code>) qaytaradi.

Muhim tushunchalar: `_like` va `as` prefikslari

- *`_like` bilan tugaydigan funksiyalar:* Bu funksiyalar (masalan, `ones_like`, `zeros_like`) mavjud massivning o'lchamlarini qo'lda yozib o'tirmasdan, aynan o'sha massiv nusxasidagi o'lcham va turni avtomatik qo'llash uchun juda qulay.
- *`as` bilan boshlanadigan funksiyalar:* Bu funksiyalar (masalan, `asarray`) xotirani tejashga yordam beradi. Agar kiruvchi ma'lumot allaqachon NumPy massivi bo'lsa, u yangi nusxa yaratmaydi, shunchaki mavjud obyektidan foydalanadi.

Bu metodlar NumPy-ning asosi bo'lib, ularni to'g'ri tanlash dasturning xotira va vaqt bo'yicha samaradorligini sezilarli darajada oshiradi.

Eslatma: Ushbu NumPy massivlarini yaratish usullari ma'lumotlar fani (data science), mashinali o'rganish (machine learning) va ilmiy hisoblashlarda (scientific computing) massivlar bilan samarali ishlashning poydevori hisoblanadi.

Nima uchun bu usullarni bilish muhim?

1. **Xotira samaradorligi:** `np.empty()` yoki `np.zeros()` kabi funksiyalar yordamida massiv uchun xotirani oldindan ajratib qo'yish (pre-allocation) dastur tezligini sezilarli darajada oshiradi.
2. **Matematik aniqlik:** `np.linspace()` va `np.logspace()` kabi metodlar ilmiy grafiklarni chizish va funksiyalarni modellashtirishda nuqtalarning aniq taqsimlanishini ta'minlaydi.
3. **Modelni ishga tushirish:** Mashinali o'rganishda neyron tarmoq vaznlarini (weights) ko'pincha tasodifiy (`np.random.rand`) yoki birlik (`np.eye`) matritsalar bilan boshlang'ich holatga keltirish talab etiladi.

NumPy massivlarini yaratishni puxta o'rganish sizga nafaqat kod yozishni, balki ma'lumotlar bilan ishlashning eng samarali yo'llarini tanlashni ham o'rgatadi.



Nazorat savollari

1-daraja: Nazariy tushunchalar (Boshlang'ich)

1. **NumPy** massivlarining oddiy Python ro'yxatlaridan (list) asosiy farqi va afzalligi nimada?
2. **“Suzuvchi nuqtali sonlar”** deganda nimani tushunasiz va NumPy-da ular qanday belgilanadi?
3. `np.array()` funksiyasiga argument sifatida qanday ma'lumot turlarini uzatish mumkin?
4. **Birlik matritsa** (Identity Matrix) nima va u qaysi funksiya yordamida yaratiladi?

2-daraja: Funksiyalar va parametrlar (O'rta)

5. `np.arange()` va `np.linspace()` funksiyalarining farqini tushuntirib bering. Qaysi holatda qadam (step), qaysi holatda elementlar soni (num) ishlatiladi?

6. `np.empty()` funksiyasi yaratgan massiv nima uchun har safar turlicha qiymatlarga ega bo'lishi mumkin? "Axlat ma'lumotlar" (garbage values) nima?
7. `order='C'` va `order='F'` parametrlarining xotirada ma'lumot joylashishiga ta'sirini izohlang.
8. `np.ones_like()` yoki `np.zeros_like()` funksiyalari nima uchun qulay? Ularning `np.ones()` va `np.zeros()` dan farqi nimada?

3-daraja: Mantiqiy va amaliy topshiriqlar (Yuqori)

9. Tasodifiy sonlar bilan ishlashda `np.random.rand()` va `np.random.randint()` funksiyalari orasidagi farqni misollar bilan ko'rsating.
10. Tuzilmali massivlar (Structured Arrays) nima uchun jadval ko'rinishidagi ma'lumotlarni saqlashda ishlatiladi? `dtype` parametrini qanday sozlash kerak?
11. Agar bizga 0 dan 1 gacha bo'lgan oraliqda logarifmik shkalada joylashgan sonlar kerak bo'lsa, qaysi funksiyadan foydalanamiz?
12. `np.diag()` funksiyasi ikki xil vazifani bajarishi mumkin. Bular qaysilar?